UNIVERSITY OF LEEDS

COMP1850 Programming

Week 4 Session 1

Functions

Recap

- So far, you have written code to perform tasks involving various data types and collections with:
  - Logical conditions
  - Repetition

- We may also want to reuse a block of code repeatedly within the same file (or, also across files or projects) :
  - Code reuse is an important design consideration
  - Related to the organisation of source code
    rather than the flow of execution

UNIVERSITY OF LEEDS

Today's aims:

To introduce functions :

- To define functions based on specifications
- To write programs with functions and parameter passing
- To write comments and docstrings to specify what a function does

UNIVERSITY OF LEEDS

What is a function?

- A function is a block of code used to perform a single, well-defined task.

- A function is a "*black-box*" defined by:

  - The name of the function
  - The purpose of the function
  - The inputs to the function
  - The outputs of the function
  - The (internal) implementation

Types of function

UNIVERSITY OF LEEDS

We can broadly categorise Python functions into:

- Built-in functions
- Built-in methods
- User-defined
  - functions or methods

UNIVERSITY OF LEEDS

Single responsibility principle (SRP)

A software design guideline that states:

"**each function in a program should have only one responsibility or purpose**"

The Python built-in functions and methods each have a well-defined single purpose

UNIVERSITY OF LEEDS

You have seen & used functions already

Python built-in functions

- `print – print("Hello world")`
- `input – name=input("What is your name?")`
- `int – number=int(input("Enter a number: "))`

UNIVERSITY OF LEEDS

Python built-in functions

True "*Black-box*" functions

https://docs.python.org/3/builtins/functions.html

Do we :

- Know the name of the function ?
- Know how the function is implemented ?
- Know the inputs required to use the function ?
- Know the outputs produced by the function ?

UNIVERSITY OF LEEDS

Task 1: input()

```
reply = input( "Tell me something : " )
```

What is:

- The purpose of the function
- Input for the function
- Output for the function

Think about data types as well as behaviour

Write a small example to test input()

UNIVERSITY OF LEEDS

You have seen & used methods already

Methods for Python built-in data structures

- `String: "hello".upper(), name.title()`
- `List: shopping.append("bananans"), shopping.sort()`
- `Tuple: fruit.index("banana"), fruit.count("cherry")`
- `etc…`

Python methods

Data such as lists, strings and tuples are objects.

Operations on those objects are provided as "methods"

- The method is specific to that type of data
- Documentation tells us which methods are available

Task 2: *set*.intersection()

```
int_set = setA.intersection( setB )
```

What is:

- The purpose of the method
- Input for the method (type)
- Output of the method (type and value)

Write a small example

- Consider the differences to built-in functions

Defining our own functions

UNIVERSITY OF LEEDS

The built-in functions and methods are part of the Python language

- They are documented so we understand their use
- They are already defined and implemented in code
- We call (use) the functions when we need them

We can write our own functions with similar principles

UNIVERSITY OF LEEDS

Writing our own functions

- We can create our own functions.
  - These are called *user-defined functions*
  - We should respect the SRP and design them carefully

- Advantages of using functions:
  - Simpler program design – through *functional decomposition*
  - Simpler testing of small units of code
  - Simpler development - can divide work among a team
  - Simpler code sharing - increased code reusability

Syntax to define a function

UNIVERSITY OF LEEDS

```
def function_name( parameter_list ):
    statement1
    statement2
    …
    return return_expression
```

UNIVERSITY OF LEEDS

Components of a function

- **def** – keyword
- **function_name** - unique meaningful name
- **parameter_list** - function inputs
- Function body – indented statements
- **return** – keyword
- **return_expression** – function output

```
def function_name( parameter_list ):
    statement1
    statement2
    …
    return return_expression
```

Task 3

Name and define a function, inputs, outputs and return type for the following tasks:

- A function that calculates and returns the area of a rectangle.
- A function that accepts an integer value and returns a list of even integers between 0 and that value.
- A function that accepts a list of integers and returns the mean average of the list.

UNIVERSITY OF LEEDS

Function definition

- A function is defined using the def keyword
  - Eg.

```
def check_vote_eligibility( age ) :
```

- The def keyword instructs the Python interpreter to create a function

- The function name should be meaningful
  - Eg. What can we infer from the function definition above?

UNIVERSITY OF LEEDS

def and the Python interpreter

- Python functions are created as objects
  - Just like other variables in your code

- def behaves like assignment (=) for functions
  - Assign a variable 'fruit': fruit = ['apple', 'orange']
  - Assign (define) a function 'check_vote': def check_vote(…):

- This property will be used in various way later on

UNIVERSITY OF LEEDS

The return statement

- The `return` statement allows the function to respond to the calling environment
  - Exits the function

- Whatever follows the `return` keyword is passed back to the calling code as the value of the function

- Any type of Python object can be returned.
  - Eg. what type and value is returned by ?
    - hi = "hello".upper()
    - choice = ['cherry', 'grapes', 'banana'].index('banana')
  - Returning no value (or omitting return) has value `None`

Documentation: Comments and docstrings

UNIVERSITY OF LEEDS

- What for? To help everyone to understand the purpose of a line or a section of code.
- Good code should be well documented
- The best form of documentation is the code itself
  - self-describing code with good descriptive names for variables and functions

UNIVERSITY OF LEEDS

Good practice for writing code comments

- Comments should not contradict the code

```
delay(200); # delay one second here
```

- Comments should not repeat exactly what is being said already by the code

```
i=i+1   # increment i by 1
```

- Do not over-comment code

Comments

- Comment added with # or """ (triple quotes)
  - Single line

```
# Check if the sanitized string is equal to its reverse
```

  - Multi-line

```
"""
Check if a given string is a palindrome. A palindrome
is a word, phrase, or sequence that reads the same
backward as forward.
"""
```

UNIVERSITY OF LEEDS

Docstrings

- First statement in a function or method
  - Should describe the function or method
  - Always use triple quotes """
- Single-line docstrings

```
def func3(a, b):
    """Return the multiplication of a and b."""
```

- Multi-line docstrings

```
def complex(real=0.0, imag=0.0):
    """
     Form a complex number.

    Keyword arguments:
    real -- the real part (default 0.0)
    imag -- the imaginary part (default 0.0)
    """
```

UNIVERSITY OF LEEDS

Return to Task 3

Document your code

- Docstrings for functions
- Inline comments if necessary

UNIVERSITY OF LEEDS

Task 4

Open the task4 folder and the `mystery.py` file.

Run the program and inspect the code and output.

Has the program produced what you expected?

Write the docstring for the function `mystery_func()`.

Inspect and run the code in `process_data.py`, fixing any errors.

Inspect and run the code in `filter.py`, correcting the docstring.

Coding functions in Python

UNIVERSITY OF LEEDS

- Because the Python interpreter creates a function with def
  - We must define a function before we can use it
    - We have been doing this already in the tasks
  - For code organisation it is better to provide all function implementations first

- The interpreter processes the Python file sequentially from line 1
  - Typically, all function definitions come first in the written code
  - There are exceptions to this and Python allows a lot of flexibility

UNIVERSITY OF LEEDS

Calling a function

- To call a function, type the name of the function followed by parentheses (), with required argument(s)

```
def add_two_numbers(num1, num2):   # defining the function
    return num1+num2                # (function environment)

result = add_two_numbers(3, 4)    # using the function
print(f'Result: {result}')       # (calling environment)
```

- We call (use) the function by providing values for the arguments
- We assign the returned value to a variable so we can use it

UNIVERSITY OF LEEDS

The calling environment

In the calling environment we pass values to the function either:

- as raw values, eg. numerical data
- as values stored in variables of any type

```
result = add_two_numbers( 4, 5 )          # values
result = add_two_numbers( number1, number2 )     # variables
```

In the function environment the parameters are assigned values from the data that is passed

In the function environment

UNIVERSITY OF LEEDS

Call-by-value model

- The data is a copy of the data from the calling environment
- Changing values in the function does not affect the calling environment

Call-by-reference model

- The data is a reference to the data in the calling environment
- Changing values in the function changes data in the calling environment

- The Python model is termed call-by-object-reference
  - The behaviour is determined by properties of the object

Call-by-object-reference

UNIVERSITY OF LEEDS

The behaviour is determined by mutability

- Recall, for example, lists are mutable, sets are not
- Simple variables and numerical values are immutable

Mutable data behaves according to call-by-reference

Immutable data behaves according to call-by-value

UNIVERSITY OF LEEDS

Further practice

Tasks 5 and 6 ask you to create functions

- Document code as you write it
- Always test the functions you create

UNIVERSITY OF LEEDS

Task 5

Open the task 5 folder and complete the tasks.

Consider the structure of the code. Identify:

- Function and calling environments
- Function inputs and outputs

Run the program and inspect the code and output.

Complete the other exercises

UNIVERSITY OF LEEDS

Task 6

- Open task3 folder and `sample2.py` file.
- Run the program and inspect the code and output.
- Is the output what you expected?

- Complete all other exercises

UNIVERSITY OF LEEDS

Next session

- Function arguments
- Default parameter values
- Functions returning multiple values
- Recursive functions